

Loan & EMI Reference

Fallback reference only — write your own logic in the real files first. This shows one complete, working way to build it, file by file. Field names are suggestions; keep both modules in sync with whatever you actually choose.

Relationship: `Emi` holds `@ManyToOne Loan loan` — `Loan` does **not** hold a `@OneToMany List<Emi>`. Same convention as `Transaction / LedgerEntry`: the child row points back to its parent, the parent entity stays thin.

Who generates the schedule: `LoanServiceImpl` orchestrates (owns the contract lifecycle: apply → approve/reject → closed), and calls into `EmiService` to generate the installment rows once a loan is approved — the same shape as `TransactionServiceImpl.transfer()` calling `ledgerService.recordTransactionEntries(transaction)`.

Avoiding a circular dependency: `LoanServiceImpl` depends on `EmiService` (to generate the schedule). When an EMI gets paid and it turns out to be the *last* one, something needs to close the loan — but `EmiServiceImpl` depending on `LoanService` back would create a circular service dependency. The fix used below: `EmiServiceImpl` depends on `LoanRepository` directly (not `LoanService`) to flip the parent's status. Both services are allowed to depend on each other's *repository*; it's depending on each other's *service* in both directions that creates the cycle.

Loan module

`loan/entity/LoanStatus.java`

```
package org.example.enterprisedigitalbankingsystem.loan.entity;

public enum LoanStatus {
    PENDING,    // applied, awaiting approval
    REJECTED,   // approval declined
    ACTIVE,     // approved + disbursed, currently repaying
    CLOSED      // every EMI paid
}
```

`loan/entity/LoanType.java`

```
package org.example.enterprisedigitalbankingsystem.loan.entity;

public enum LoanType {
    PERSONAL,
    HOME,
    VEHICLE,
    EDUCATION
}
```

`loan/entity/Loan.java`

```

package org.example.enterprisedigitalbankingsystem.loan.entity;

import jakarta.persistence.*;
import lombok.*;
import org.example.enterprisedigitalbankingsystem.account.entity.Account;
import org.example.enterprisedigitalbankingsystem.customer.entity.Customer;

import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "loans")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Loan {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "customer_id", nullable = false)
    private Customer customer;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "disbursement_account_id", nullable = false)
    private Account disbursementAccount;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private LoanType loanType;

    @Column(nullable = false, precision = 18, scale = 2)
    private BigDecimal principalAmount;

    @Column(nullable = false, precision = 5, scale = 2)
    private BigDecimal interestRate;    // annual percentage, e.g. 10.50

    @Column(nullable = false)
    private Integer tenureMonths;

    @Column(nullable = false, precision = 18, scale = 2)
    private BigDecimal emiAmount;    // computed at application time

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private LoanStatus status;

    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt;

    private LocalDateTime approvedAt;
    private LocalDateTime closedAt;

    @PrePersist
    public void prePersist() {

```

```
        createdAt = LocalDateTime.now();  
        if (status == null) status = LoanStatus.PENDING;  
    }  
}
```

loan/repository/LoanRepository.java

```
package org.example.enterprisedigitalbankingsystem.loan.repository;  
  
import org.example.enterprisedigitalbankingsystem.loan.entity.Loan;  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import java.util.List;  
  
public interface LoanRepository extends JpaRepository<Loan, Long> {  
    List<Loan> findByCustomerId(Long customerId);  
}
```

loan/dto/request/CreateLoanRequest.java

```

package org.example.enterprisedigitalbankingsystem.loan.dto.request;

import jakarta.validation.constraints.*;
import lombok.*;
import org.example.enterprisedigitalbankingsystem.loan.entity.LoanType;

import java.math.BigDecimal;

@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class CreateLoanRequest {

    @NotNull(message = "Customer id is required")
    private Long customerId;

    @NotNull(message = "Disbursement account id is required")
    private Long disbursementAccountId;

    @NotNull(message = "Loan type is required")
    private LoanType loanType;

    @NotNull(message = "Principal amount is required")
    @DecimalMin(value = "1000.00", message = "Principal must be at least 1000")
    private BigDecimal principalAmount;

    @NotNull(message = "Interest rate is required")
    @DecimalMin(value = "0.01", message = "Interest rate must be positive")
    private BigDecimal interestRate;

    @NotNull(message = "Tenure is required")
    @Min(value = 1, message = "Tenure must be at least 1 month")
    @Max(value = 360, message = "Tenure cannot exceed 360 months")
    private Integer tenureMonths;
}

```

loan/dto/response/LoanResponse.java

```

package org.example.enterprisedigitalbankingsystem.loan.dto.response;

import lombok.*;
import org.example.enterprisedigitalbankingsystem.loan.entity.LoanStatus;
import org.example.enterprisedigitalbankingsystem.loan.entity.LoanType;

import java.math.BigDecimal;
import java.time.LocalDateTime;

@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class LoanResponse {
    private Long id;
    private Long customerId;
    private String disbursementAccountNumber;
    private LoanType loanType;
    private BigDecimal principalAmount;
    private BigDecimal interestRate;
    private Integer tenureMonths;
    private BigDecimal emiAmount;
    private LoanStatus status;
    private LocalDateTime createdAt;
    private LocalDateTime approvedAt;
    private LocalDateTime closedAt;
}

```

loan/mapper/LoanMapper.java

```

package org.example.enterprisedigitalbankingsystem.loan.mapper;

import org.example.enterprisedigitalbankingsystem.loan.dto.response.LoanResponse;
import org.example.enterprisedigitalbankingsystem.loan.entity.Loan;
import org.springframework.stereotype.Component;

@Component
public class LoanMapper {
    public LoanResponse toResponse(Loan loan) {
        if (loan == null) return null;
        return LoanResponse.builder()
            .id(loan.getId())
            .customerId(loan.getCustomer().getId())
            .disbursementAccountNumber(loan.getDisbursementAccount().getAccountNumber())
            .loanType(loan.getLoanType())
            .principalAmount(loan.getPrincipalAmount())
            .interestRate(loan.getInterestRate())
            .tenureMonths(loan.getTenureMonths())
            .emiAmount(loan.getEmiAmount())
            .status(loan.getStatus())
            .createdAt(loan.getCreatedAt())
            .approvedAt(loan.getApprovedAt())
            .closedAt(loan.getClosedAt())
            .build();
    }
}

```

loan/service/LoanService.java

```
package org.example.enterprisedigitalbankingsystem.loan.service;

import org.example.enterprisedigitalbankingsystem.loan.dto.request.CreateLoanRequest;
import org.example.enterprisedigitalbankingsystem.loan.dto.response.LoanResponse;

import java.util.List;

public interface LoanService {
    LoanResponse applyForLoan(CreateLoanRequest request);
    LoanResponse getLoanById(Long loanId);
    List<LoanResponse> getLoansByCustomerId(Long customerId);
    LoanResponse approveLoan(Long loanId);
    LoanResponse rejectLoan(Long loanId);
}
```

loan/service/impl/LoanServiceImpl.java

```

package org.example.enterprisedigitalbankingsystem.loan.service.impl;

import lombok.RequiredArgsConstructor;
import org.example.enterprisedigitalbankingsystem.account.entity.Account;
import org.example.enterprisedigitalbankingsystem.account.repository.AccountRepository;
import org.example.enterprisedigitalbankingsystem.customer.entity.Customer;
import org.example.enterprisedigitalbankingsystem.customer.repository.CustomerRepository;
import org.example.enterprisedigitalbankingsystem.emi.service.EmiService;
import org.example.enterprisedigitalbankingsystem.exception.BadRequestException;
import org.example.enterprisedigitalbankingsystem.exception.ResourceNotFoundException;
import org.example.enterprisedigitalbankingsystem.loan.dto.request.CreateLoanRequest;
import org.example.enterprisedigitalbankingsystem.loan.dto.response.LoanResponse;
import org.example.enterprisedigitalbankingsystem.loan.entity.Loan;
import org.example.enterprisedigitalbankingsystem.loan.entity.LoanStatus;
import org.example.enterprisedigitalbankingsystem.loan.mapper.LoanMapper;
import org.example.enterprisedigitalbankingsystem.loan.repository.LoanRepository;
import org.example.enterprisedigitalbankingsystem.loan.service.LoanService;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.util.List;

@Service
@RequiredArgsConstructor
@Transactional
public class LoanServiceImpl implements LoanService {

    private final LoanRepository loanRepository;
    private final CustomerRepository customerRepository;
    private final AccountRepository accountRepository;
    private final LoanMapper loanMapper;
    private final EmiService emiService; // the collaborator – generates the schedule

    @Override
    public LoanResponse applyForLoan(CreateLoanRequest request) {
        Customer customer = customerRepository.findById(request.getCustomerId())
            .orElseThrow(() -> new ResourceNotFoundException(
                "Customer not found with id: " + request.getCustomerId()));

        Account account = accountRepository.findById(request.getDisbursementAccountId())
            .orElseThrow(() -> new ResourceNotFoundException(
                "Account not found with id: " + request.getDisbursementAccountId()));

        BigDecimal emiAmount = calculateEmi(
            request.getPrincipalAmount(), request.getInterestRate(), request.getTenureMonths());

        Loan loan = Loan.builder()
            .customer(customer)
            .disbursementAccount(account)
            .loanType(request.getLoanType())
            .principalAmount(request.getPrincipalAmount())
            .interestRate(request.getInterestRate())

```

```

        .tenureMonths(request.getTenureMonths())
        .emiAmount(emiAmount)
        .status(LoanStatus.PENDING)
        .build();

    loan = loanRepository.save(loan);
    return loanMapper.toResponse(loan);
}

@Override
@Transactional(readOnly = true)
public LoanResponse getLoanById(Long loanId) {
    return loanMapper.toResponse(findLoanById(loanId));
}

@Override
@Transactional(readOnly = true)
public List<LoanResponse> getLoansByCustomerId(Long customerId) {
    if (!customerRepository.existsById(customerId)) {
        throw new ResourceNotFoundException("Customer not found with id: " + customerId);
    }
    return loanRepository.findByCustomerId(customerId).stream()
        .map(loanMapper::toResponse)
        .toList();
}

@Override
public LoanResponse approveLoan(Long loanId) {
    Loan loan = findLoanById(loanId);
    if (loan.getStatus() != LoanStatus.PENDING) {
        throw new BadRequestException("Loan is " + loan.getStatus() + " and cannot be approved");
    }

    // Disburse: credit the loan amount into the customer's account.
    Account account = loan.getDisbursementAccount();
    account.setBalance(account.getBalance().add(loan.getPrincipalAmount()));
    accountRepository.save(account);

    loan.setStatus(LoanStatus.ACTIVE);
    loan.setApprovedAt(LocalDateTime.now());
    loan = loanRepository.save(loan);

    emiService.generateSchedule(loan); // builds every Emi row for this loan

    return loanMapper.toResponse(loan);
}

@Override
public LoanResponse rejectLoan(Long loanId) {
    Loan loan = findLoanById(loanId);
    if (loan.getStatus() != LoanStatus.PENDING) {
        throw new BadRequestException("Loan is " + loan.getStatus() + " and cannot be rejected");
    }

    loan.setStatus(LoanStatus.REJECTED);
    loan = loanRepository.save(loan);
}

```

```

        return loanMapper.toResponse(loan);
    }

    /**
     * Standard reducing-balance EMI formula:
     *  $EMI = P * r * (1+r)^n / ((1+r)^n - 1)$ 
     * where  $r$  = monthly interest rate ( $annual\% / 12 / 100$ ),  $n$  = tenure in months.
     */
    private BigDecimal calculateEmi(BigDecimal principal, BigDecimal annualRatePercent, int tenureMonths) {
        double p = principal.doubleValue();
        double r = annualRatePercent.doubleValue() / 12 / 100;
        double factor = Math.pow(1 + r, tenureMonths);
        double emi = p * r * factor / (factor - 1);
        return BigDecimal.valueOf(emi).setScale(2, RoundingMode.HALF_UP);
    }

    private Loan findLoanById(Long loanId) {
        return loanRepository.findById(loanId)
            .orElseThrow(() -> new ResourceNotFoundException("Loan not found with id: " + loanId));
    }
}

```

Note: the EMI math is done in `double` purely for the $(1+r)^n$ exponent — `BigDecimal` has no built-in fractional-exponent power function. The *result* is immediately rounded back into a 2-decimal `BigDecimal` for storage, so no imprecise `double` value is ever persisted or returned to a client.

loan/controller/LoanController.java

```

package org.example.enterprisedigitalbankingsystem.loan.controller;

import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.example.enterprisedigitalbankingsystem.loan.dto.request.CreateLoanRequest;
import org.example.enterprisedigitalbankingsystem.loan.dto.response.LoanResponse;
import org.example.enterprisedigitalbankingsystem.loan.service.LoanService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1/loans")
@RequiredArgsConstructor
public class LoanController {

    private final LoanService loanService;

    @PostMapping
    public ResponseEntity<LoanResponse> applyForLoan(@Valid @RequestBody CreateLoanRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED).body(loanService.applyForLoan(request));
    }

    @GetMapping("/{loanId}")
    public ResponseEntity<LoanResponse> getLoanById(@PathVariable Long loanId) {
        return ResponseEntity.ok(loanService.getLoanById(loanId));
    }

    @GetMapping("/customer/{customerId}")
    public ResponseEntity<List<LoanResponse>> getLoansByCustomerId(@PathVariable Long customerId) {
        return ResponseEntity.ok(loanService.getLoansByCustomerId(customerId));
    }

    @PatchMapping("/{loanId}/approve")
    public ResponseEntity<LoanResponse> approveLoan(@PathVariable Long loanId) {
        return ResponseEntity.ok(loanService.approveLoan(loanId));
    }

    @PatchMapping("/{loanId}/reject")
    public ResponseEntity<LoanResponse> rejectLoan(@PathVariable Long loanId) {
        return ResponseEntity.ok(loanService.rejectLoan(loanId));
    }
}

```

EMI module

emi/entity/EmiStatus.java

```

package org.example.enterprisedigitalbankingsystem.emi.entity;

public enum EmiStatus {
    PENDING,
    PAID,
    OVERDUE
}

```

emi/entity/Emi.java

```

package org.example.enterprisedigitalbankingsystem.emi.entity;

import jakarta.persistence.*;
import lombok.*;
import org.example.enterprisedigitalbankingsystem.loan.entity.Loan;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.LocalDateTime;

@Entity
@Table(name = "emis")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Emi {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "loan_id", nullable = false)
    private Loan loan;

    @Column(nullable = false)
    private Integer installmentNumber; // 1, 2, 3 ... tenureMonths

    @Column(nullable = false)
    private LocalDate dueDate;

    @Column(nullable = false, precision = 18, scale = 2)
    private BigDecimal amount;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private EmiStatus status;

    private LocalDateTime paidAt;

    @PrePersist
    public void prePersist() {
        if (status == null) status = EmiStatus.PENDING;
    }
}

```

emi/repository/EmiRepository.java

```
package org.example.enterprisedigitalbankingsystem.emi.repository;

import org.example.enterprisedigitalbankingsystem.emi.entity.Emi;
import org.example.enterprisedigitalbankingsystem.emi.entity.EmiStatus;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface EmiRepository extends JpaRepository<Emi, Long> {
    List<Emi> findByLoanIdOrderByInstallmentNumberAsc(Long loanId);
    boolean existsByLoanIdAndStatusNot(Long loanId, EmiStatus status);
}
```

`existsByLoanIdAndStatusNot(loanId, PAID)` is the trick used below to check "does this loan have any unpaid EMI left" in one query, instead of loading every row just to check their statuses in Java.

emi/dto/response/EmiResponse.java

```
package org.example.enterprisedigitalbankingsystem.emi.dto.response;

import lombok.*;
import org.example.enterprisedigitalbankingsystem.emi.entity.EmiStatus;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.LocalDateTime;

@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class EmiResponse {
    private Long id;
    private Long loanId;
    private Integer installmentNumber;
    private LocalDate dueDate;
    private BigDecimal amount;
    private EmiStatus status;
    private LocalDateTime paidAt;
}
```

No `CreateEmiRequest` is needed — EMI rows are never submitted by a client, only generated by `EmiService.generateSchedule(Loan loan)` when a loan is approved. The empty `CreateEmiRequest.java` file can just stay unused, or be deleted.

emi/mapper/EmiMapper.java

```

package org.example.enterprisedigitalbankingsystem.emi.mapper;

import org.example.enterprisedigitalbankingsystem.emi.dto.response.EmiResponse;
import org.example.enterprisedigitalbankingsystem.emi.entity.Emi;
import org.springframework.stereotype.Component;

@Component
public class EmiMapper {
    public EmiResponse toResponse(Emi emi) {
        if (emi == null) return null;
        return EmiResponse.builder()
            .id(emi.getId())
            .loanId(emi.getLoan().getId())
            .installmentNumber(emi.getInstallmentNumber())
            .dueDate(emi.getDueDate())
            .amount(emi.getAmount())
            .status(emi.getStatus())
            .paidAt(emi.getPaidAt())
            .build();
    }
}

```

emi/service/EmiService.java

```

package org.example.enterprisedigitalbankingsystem.emi.service;

import org.example.enterprisedigitalbankingsystem.emi.dto.response.EmiResponse;
import org.example.enterprisedigitalbankingsystem.loan.entity.Loan;

import java.util.List;

public interface EmiService {
    void generateSchedule(Loan loan);
    List<EmiResponse> getEmisByLoanId(Long loanId);
    EmiResponse payEmi(Long emiId);
}

```

emi/service/impl/EmiServiceImpl.java

```

package org.example.enterprisedigitalbankingsystem.emi.service.impl;

import lombok.RequiredArgsConstructor;
import org.example.enterprisedigitalbankingsystem.emi.dto.response.EmiResponse;
import org.example.enterprisedigitalbankingsystem.emi.entity.Emi;
import org.example.enterprisedigitalbankingsystem.emi.entity.EmiStatus;
import org.example.enterprisedigitalbankingsystem.emi.mapper.EmiMapper;
import org.example.enterprisedigitalbankingsystem.emi.repository.EmiRepository;
import org.example.enterprisedigitalbankingsystem.emi.service.EmiService;
import org.example.enterprisedigitalbankingsystem.exception.BadRequestException;
import org.example.enterprisedigitalbankingsystem.exception.ResourceNotFoundException;
import org.example.enterprisedigitalbankingsystem.loan.entity.Loan;
import org.example.enterprisedigitalbankingsystem.loan.entity.LoanStatus;
import org.example.enterprisedigitalbankingsystem.loan.repository.LoanRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Service
@RequiredArgsConstructor
@Transactional
public class EmiServiceImpl implements EmiService {

    private final EmiRepository emiRepository;
    private final LoanRepository loanRepository; // NOT LoanService – avoids the circular dependency
    private final EmiMapper emiMapper;

    @Override
    public void generateSchedule(Loan loan) {
        List<Emi> schedule = new ArrayList<>();
        LocalDate dueDate = LocalDate.now();

        for (int installment = 1; installment <= loan.getTenureMonths(); installment++) {
            dueDate = dueDate.plusMonths(1);
            schedule.add(Emi.builder()
                .loan(loan)
                .installmentNumber(installment)
                .dueDate(dueDate)
                .amount(loan.getEmiAmount())
                .status(EmiStatus.PENDING)
                .build());
        }
        emiRepository.saveAll(schedule);
    }

    @Override
    @Transactional(readOnly = true)
    public List<EmiResponse> getEmisByLoanId(Long loanId) {
        return emiRepository.findByLoanIdOrderByInstallmentNumberAsc(loanId).stream()
            .map(emiMapper::toResponse)
            .toList();
    }

```

```

    }

    @Override
    public EmiResponse payEmi(Long emiId) {
        Emi emi = emiRepository.findById(emiId)
            .orElseThrow(() -> new ResourceNotFoundException("EMI not found with id: " + emiId));

        if (emi.getStatus() == EmiStatus.PAID) {
            throw new BadRequestException("EMI is already paid");
        }

        emi.setStatus(EmiStatus.PAID);
        emi.setPaidAt(LocalDateTime.now());
        emi = emiRepository.save(emi);

        closeLoanIfFullyPaid(emi.getLoan());

        return emiMapper.toResponse(emi);
    }

    private void closeLoanIfFullyPaid(Loan loan) {
        boolean anyUnpaidLeft = emiRepository.existsByLoanIdAndStatusNot(loan.getId(), EmiStatus.PAID);
        if (!anyUnpaidLeft) {
            loan.setStatus(LoanStatus.CLOSED);
            loan.setClosedAt(LocalDateTime.now());
            loanRepository.save(loan);
        }
    }
}

```

This is also where real-world scope would grow (not included here, deliberately kept minimal): `payEmi` doesn't actually move money out of the customer's account or create a `Transaction / LedgerEntry` pair — a fuller version would call into `TransactionService` the same way `activateBeneficiary` calls into `BeneficiaryEventProducer`, debiting the customer's account for the EMI amount before marking the installment `PAID`.

`emi/controller/EmiController.java`

```

package org.example.enterprisedigitalbankingsystem.emi.controller;

import lombok.RequiredArgsConstructor;
import org.example.enterprisedigitalbankingsystem.emi.dto.response.EmiResponse;
import org.example.enterprisedigitalbankingsystem.emi.service.EmiService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1/emis")
@RequiredArgsConstructor
public class EmiController {

    private final EmiService emiService;

    @GetMapping("/loan/{loanId}")
    public ResponseEntity<List<EmiResponse>> getEmisByLoanId(@PathVariable Long loanId) {
        return ResponseEntity.ok(emiService.getEmisByLoanId(loanId));
    }

    @PatchMapping("/{emiId}/pay")
    public ResponseEntity<EmiResponse> payEmi(@PathVariable Long emiId) {
        return ResponseEntity.ok(emiService.payEmi(emiId));
    }
}

```

Known simplifications in this reference

- `payEmi` doesn't move real money (no `Transaction` / `LedgerEntry` / account debit) — only flips the `Emi` status. Add that integration if you want paying an EMI to actually affect the customer's balance.
- No audit logging calls shown here — add `auditService.log(...)` at the same points as every other module (loan applied, approved, rejected; EMI paid), matching the convention everywhere else in this project.
- No overdue-detection job (something that flips `PENDING` → `OVERDUE` once `dueDate` has passed) — would need a scheduled task, not covered here.